

Минобрнауки России
Санкт-Петербургский государственный политехнический университет
Факультет технической кибернетики
Кафедра «Информационные и управляющие системы»

Курсовая работа

Программирование на ассемблере

по дисциплине: «ЭВМ и периферийные устройства»

Выполнил студент гр. 2084/3



Овчаров А. И.

Руководитель

доцент, к.т.н.



Молодяков С.А.

Зачи

"6" декабря 2011 г.

Санкт-Петербург

2011

Оглавление

Оглавление

Оглавление	2
Введение	2
Достоинства и недостатки	2
Достоинства	2
Недостатки	3
Цели	3
Программа 1	3
Основные понятия	3
Код	4
Скриншоты	5
Программа 2	6
Основные понятия	6
Код	7
Скриншоты	10

Введение

Язык ассемблера — язык программирования низкого уровня, мнемонические команды которого (за редким исключением) соответствуют инструкциям процессора вычислительной системы. Трансляция программы в исполняемый машинный код производится ассемблером (от англ. assembler — сборщик) — программой-транслятором, которая и дала языку ассемблера его название.

Команды языка ассемблера один к одному соответствуют командам процессора. Фактически, они и представляют собой более удобную для человека символьную форму записи — мнемокоды — команд и их аргументов. При этом одной команде языка ассемблера может соответствовать несколько вариантов команд процессора.

Достоинства и недостатки

Достоинства

Язык ассемблера позволяет писать самый быстрый и компактный код, какой вообще возможен для данного процессора.

Если код программы достаточно большой, — данные, которыми он оперирует, не помещаются целиком в регистрах процессора, то есть частично или полностью находятся в оперативной памяти, то искусный программист, как правило, способен значительно оптимизировать программу по сравнению с транслятором с языка высокого уровня по одному или нескольким параметрам:

скорость работы — за счёт оптимизации вычислительного алгоритма и/или более рационального обращения к ОП, перераспределения данных;
Обеспечение максимального использования специфических возможностей конкретной платформы, что также позволяет создавать более эффективные программы, — в том числе менее ресурсоемкие.

При программировании на языке ассемблера возможен непосредственный доступ к аппаратуре, и, в частности:

портам ввода-вывода,

регистрам процессора и др.

Язык ассемблера используется для создания «прошивок» BIOS.

С помощью дизассемблера позволяет исследовать существующие программы при отсутствии исходного кода.

Недостатки

Сложную программу написать на языке ассемблера намного сложнее — вплоть до невозможности, по причине сложности, — чем на языке высокого уровня.

В силу машинной ориентации — «низкого» уровня — языка ассемблера человеку сложнее читать и понимать программу на нём по сравнению с программой на языке высокого уровня. А подлежащий редактированию ассемблерный код сравнительно большой, — программа на языке ассемблера состоит из слишком «мелких» элементов — машинных команд. Соответственно, усложняются программирование и отладка, растёт трудоёмкость разработки, сравнительно велика вероятность программных ошибок.

Как правило, меньшее количество доступных библиотек по сравнению с современными индустриальными языками программирования.

Программы на языке ассемблера не переносимы на компьютеры с другой архитектурой и системой команд как на уровне машинных команд так и на уровне исходных кодов, что в случае с языками ассемблера одно и то же.

Цели

- 1) Разработать процедуру слияния двух упорядоченных списков вида «ФАМИЛИЯ И.О.» с сохранением алфавитного порядка. Исходные и конечный списки вывести на экран.
- 2) Мультизадачность 2. Организуйте мультизадачную среду (3 задачи) с выводом на экран номера задачи. Переключение происходит по алгоритму кругового планирования с выделенными квантами времени.

Программа 1

Основные понятия

Список — это абстрактный тип данных, представляющий собой упорядоченный набор значений, в котором некоторое значение может встречаться более одного раза. Для сортировки нового списка используем первую букву фамилии.

Kod

```
DOSSEG
.model TINY
.data

array db "IVANOV V.Y.",10,13
      db "PETROV R.A.",10,13
      db "KERGAKOV A.A.",10,13
      db "KARPIN V.G.",10,13
      db "ARSHAVIN A.S.",10,13
      db "SYCHEV D.Y.",10,13
      db "KOLYANOV I.V.",10,13
      db "KIRYAKOV S.V.",10,13
      db "MOSTOVOI A.V.",10,13
      db "SIDOROV I.V.",10,13
      db "$"

array2 db "SHUNIN A.V.",10,13
       db "AKINFEEV I.V.",10,13
       db "MALAFEEV V.A.",10,13
       db "$"

string db "-----$",10,13
.code

; swap funtion
swap:
pop dx
pop bx ; bx = &array
push dx
mov cx, 15d ; cx = num of loop
for:
push bx
add bx, cx
dec bx
mov al, [bx] ; ax = first word
mov dl, [bx+15d] ; dx = second word
mov [bx], dl
mov [bx+15d], al
pop bx
loop for
ret ; end of swap function

start:
mov ax, @data
mov ds, ax

mov bx, offset array
mov di, offset array2

add bx,150d ; конец массива

mov dx,[di] ; dx = word of array2
mov [bx], dx ; [bx]= last word of array

;print new list
mov dx, offset array
mov ah, 9h
int 21h
```

```

    mov dx, offset string
    mov ah, 9h
    int 21h

mov cx, 12d
loop2:
push cx
mov bx, offset array

mov cx, 12d
mov di, offset array
mov si, 14
loop1:

mov al, [bx+15d]
mov ah, [bx]
cmp al, ah ; сравниваем количество букв
je m1      ; если равно
cmp al, ah ; сравниваем количество букв
ja inc_str ; если выше (больше)
mov bx, di
push bx
push cx
push bx
call swap
pop cx
pop bx
mov di, bx
jmp inc_str
m1:
add bx, 1d
sub si, 1
cmp si, 0
jne loop1

jmp inc_str
inc_str:
mov bx, di
add bx, 15d
mov di, bx
loop loop1

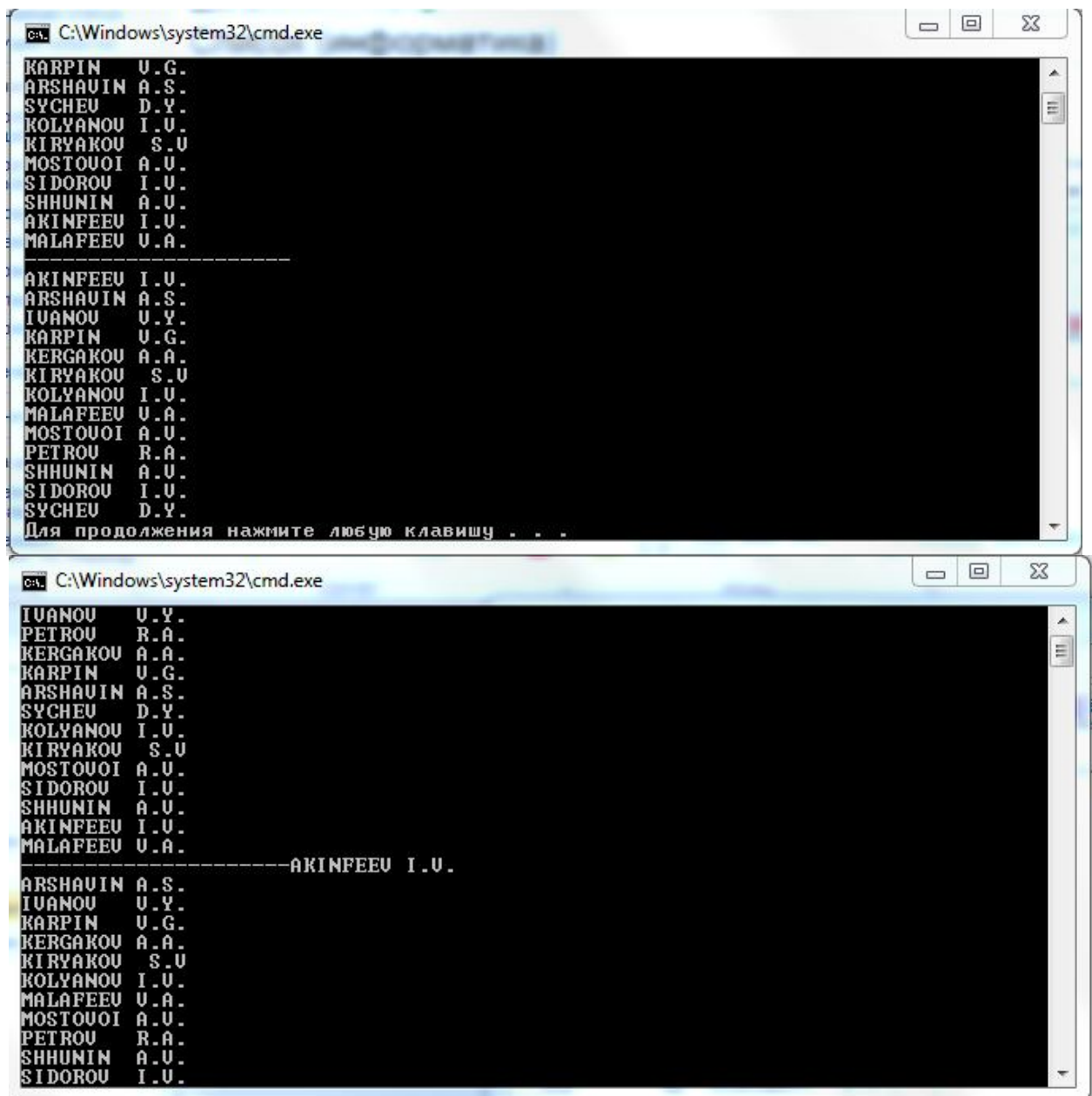
pop cx
loop loop2

mov dx, offset array
mov ah, 9h
int 21h
mov ah, 4ch
int 21h

end start

```

Скриншоты



```
C:\Windows\system32\cmd.exe
KARPIN      U.G.
ARSHAVIN    A.S.
SYCHEV      D.Y.
KOLYANOV    I.U.
KIRYAKOV    S.U.
MOSTOVOI    A.U.
SIDOROV     I.U.
SHHUNIN     A.U.
AKINFEEV    I.U.
MALAFEEV    U.A.
-----
AKINFEEV    I.U.
ARSHAVIN    A.S.
IVANOV      U.Y.
KARPIN      U.G.
KERGAKOV    A.A.
KIRYAKOV    S.U.
KOLYANOV    I.U.
MALAFEEV    U.A.
MOSTOVOI    A.U.
PETROV      R.A.
SHHUNIN     A.U.
SIDOROV     I.U.
SYCHEV      D.Y.
Для продолжения нажмите любую клавишу . . .

C:\Windows\system32\cmd.exe
IVANOV      U.Y.
PETROV      R.A.
KERGAKOV    A.A.
KARPIN      U.G.
ARSHAVIN    A.S.
SYCHEV      D.Y.
KOLYANOV    I.U.
KIRYAKOV    S.U.
MOSTOVOI    A.U.
SIDOROV     I.U.
SHHUNIN     A.U.
AKINFEEV    I.U.
MALAFEEV    U.A.
-----AKINFEEV I.U.
ARSHAVIN    A.S.
IVANOV      U.Y.
KARPIN      U.G.
KERGAKOV    A.A.
KIRYAKOV    S.U.
KOLYANOV    I.U.
MALAFEEV    U.A.
MOSTOVOI    A.U.
PETROV      R.A.
SHHUNIN     A.U.
SIDOROV     I.U.
```

Программа 2

Основные понятия

Использовался алгоритм кругового планирования. При нём таймер генерирует прерывания через определённые интервалы времени. При каждом прерывании исполняющийся в настоящий момент процесс перемещается в очередь готовых к исполнению процессов и начинает выполняться очередной процесс, выбираемый в соответствии со стратегией FCFS(first-come-first-served или схема «строгой очередности»). Эта методика известна также как "квантование времени", поскольку перед тем как быть вытесненным процесс получает кванту времени для исполнения. При круговом планировании следует избегать очень коротких квантов времени т.к. при этом возрастают накладные расходы, связанные с обработкой прерывания и выполнения функций планирования. Поэтому применимо к данному методу существует правило, которое

звучит так: квант времени должен быть немного меньше, чем время, требующееся для типичного полного обслуживания. Если квант окажется меньшего размера, то большинство процессов потребуют как минимум два кванта времени.

Код

MODEL Tiny

.386

.CODE

```
thread_struct struc ; ñòðóéòóðà, äää çàìññùääàì ðääññòðù äëý ïàñðíäðàìì
```

```
_ax dw ? ;0
```

```
_bx dw ? ;2
```

```
_cx dw ? ;4
```

```
_dx dw ? ;6
```

```
_ds dw ? ;18
```

```
_es dw ? ;20
```

```
thread_struct ends
```

```
thread1 thread_struct <>
```

```
thread2 thread_struct <>
```

```
thread3 thread_struct <>
```

```
current_thread dw 0 ; ïíäð çääà÷è
```

ORG 100h

```
old_obrab dd 0 ; ñòäðúé äâéòîð
```

```
rsi dw 1 ; ñ÷àð÷èè
```

```
rdi dw 2 ; äëý ïäðâéèþ÷áíëý çääà÷
```

task1 proc

```
;çääääàì ïà÷äëùíüä ðääññòðù èç ñòðóéòóðù
```

```
mov ax,thread1._ax
```

```
mov bx,thread1._bx
```

```
mov cx, thread1._cx
```

```
mov dx, thread1._dx
```

```
mov ds, thread1._ds
```

```
mov es, thread1._es
```

```
mov al,'a' ; äëý âùâíäà 'a'
```

```
;çääðóæääì ñíððáí,ííüä ðääññòðù
```

```
mov thread1._ax, ax
```

```
mov thread1._bx, bx
```

```
mov thread1._cx, cx
```

```
mov thread1._dx, dx
```

```
mov thread1._ds, ds
```

```
mov thread1._es, es
```

ret

task1 endp

task2 proc

```
; ñíððáíýàì ðääññòðù
```

```
mov ax,thread2._ax
```

```
mov bx,thread2._bx
```

```
mov cx, thread2._cx
```

```
mov dx, thread2._dx
```

```
mov ds, thread2._ds
```

```
mov es, thread2._es
```

```

mov al,'b'      ; äëÿ âûâîää 'b'

;çääðóæàâî ñîððáí,ííûå ðääèñòðû
mov thread2._ax, ax
mov thread2._bx, bx
mov thread2._cx, cx
mov thread2._dx, dx
mov thread2._ds, ds
mov thread2._es, es
ret
task2 endp

```

```

task3 proc
; ñîððáíÿâî ðääèñòðû
mov ax,thread3._ax
mov bx,thread3._bx
mov cx, thread3._cx
mov dx, thread3._dx
mov ds, thread3._ds
mov es, thread3._es

mov al,'c'      ; äëÿ âûâîää 'c'

;çääðóæàâî ñîððáí,ííûå ðääèñòðû
mov thread3._ax, ax
mov thread3._bx, bx
mov thread3._cx, cx
mov thread3._dx, dx
mov thread3._ds, ds
mov thread3._es, es
ret
task3 endp

```

```

new_obrab proc far

    mov si,rsi
    mov di,rdi
    add di, 1
    cmp di, 18
    jne print
    mov di, 0
    add si,1
    cmp si,4
    je obnul
    jmp print
obnul:
    mov si,1
    jmp print

print:
    mov rsi,si
    mov rdi,di
    mov current_thread,si

    pusha
    cmp si,1
    je m1
    cmp si,2
    je m2

```

```

    cmp si,3
    je m3
m1:
    call task1
    jmp m4
m2:
    call task2
    jmp m4
m3:
    call task3
    jmp m4
m4:

    ;mov ax,3    ; äëÿ î÷ëñòèè ýêðàíà
    ;int 10h

    int      29h      ; Âûâîäè 'a' , 'b' èëè 'c'
    popa
    mov al,20h
    out 20h,al

```

```

    iret
new_obrah endp

```

Start:

```

    mov ah,35h
    mov al,08h
    int 21h
    mov word ptr old_obrah+2,es
    mov word ptr old_obrah,bx
    push cs
    pop ds
    mov dx,offset new_obrah
    mov ah,25h
    mov al,08h
    int 21h

```

```

    mov thread1._ax, ax
    mov thread1._bx, bx
    mov thread1._cx, cx
    mov thread1._dx, dx
    mov thread1._ds, ds
    mov thread1._es, es

```

```

    mov thread2._ax, ax
    mov thread2._bx, bx
    mov thread2._cx, cx
    mov thread2._dx, dx
    mov thread2._ds, ds
    mov thread2._es, es

```

```

    mov thread3._ax, ax
    mov thread3._bx, bx
    mov thread3._cx, cx
    mov thread3._dx, dx
    mov thread3._ds, ds
    mov thread3._es, es

```

```

    mov current_thread,1

```

END Start

Скриншоты

